Parallel Prime Number Generator using Multithreading

A Course Project Report
Submitted in partial fulfillment of the completion of the course
23CS302PC303 – High Performance Computing

In III Year II Semester B.Tech – CSE

Submitted by
Konda Nithish Raj
Hall Ticket Number: 2303A510G4
Under the supervision of
Dr. Sharan Bediga
Assistant Professor, SOCS & AI

Academic Year 2025–2026

Department of Computer Science & Artificial Intelligence
SR University
Warangal, Telangana, India

INDEX

1. Introduction

Prime numbers are one of the most important concepts in mathematics and computer science. They are widely used in modern applications such as cryptography, data security, digital signatures, hashing techniques, and scientific computations. Generating prime numbers efficiently becomes essential when working with large numerical ranges, especially in systems that require high computational performance.

Traditionally, prime number generation is performed using serial computation, where each number is checked one after another to determine whether it is prime. While this approach works well for smaller ranges, it becomes time-consuming and inefficient when the size of the input increases. As modern computers are equipped with multicore processors, relying only on sequential execution does not fully utilize the available hardware resources.

This project focuses on the implementation of a Parallel Prime Number Generator using multithreading. The project compares the traditional serial approach with a multithreaded implementation to analyze performance improvements achieved through parallel execution.

Prime Number Generation Concept

A prime number is defined as a natural number greater than 1 that has only two positive divisors: 1 and itself. Examples of prime numbers include 2, 3, 5, 7, 11, and 13. Determining whether a number is prime involves checking if it is divisible by any number other than 1 and itself.

In computational terms, prime number generation requires repeatedly performing divisibility checks for each number in a given range. As the range grows larger, the number of required calculations increases rapidly, making the process computationally intensive.

Parallel processing helps overcome this limitation by distributing the checking process across multiple threads, allowing several numbers to be evaluated simultaneously instead of sequentially.

Input Representation

For this project, instead of using predefined datasets, numbers are generated dynamically based on user input. The program accepts a numerical range within which prime numbers need to be identified and processed.

The input is provided directly by the user during program execution. The system generates all numbers within the specified range and performs prime number verification on each value.

Each input consists of:

- A starting value of the numerical range
- An ending value of the numerical range

Example format of the input:

```
Start Range: 1
End Range: 100
```

During program execution, the given range is divided into smaller subranges, and each subrange is assigned to separate threads for parallel processing. Every thread independently checks the numbers in its assigned range to determine whether they are prime.

This input representation simulates large-scale computational workloads and enables the implementation and comparison of serial and multithreaded prime number generation approaches.

2. Problem Statement

The main objective of this project is to efficiently generate prime numbers within a given numerical range using parallel computing techniques.

The problem involves the following tasks:

- Accepting a numerical range as input from the user
- Checking each number within the range to determine whether it is prime
- Generating and displaying all prime numbers in the specified range
- Measuring execution time for different computational approaches

Challenges in the Problem

One of the major challenges in this problem is handling large numerical ranges efficiently. When the range of numbers increases significantly, checking each number sequentially becomes computationally expensive and time-consuming.Prime number generation requires performing multiple divisibility checks for every number. As the input size grows, the amount of computation increases rapidly, leading to performance limitations in serial execution.

The time complexity of DNA matching can be expressed as:

$$O(n \times \sqrt{n})$$

where:

- n = total numbers in the given range
- m = number of divisibility checks required for each number

As the range size increases, total execution time grows considerably, making serial computation inefficient for large-scale problems.

Need for High Performance Computing

To overcome these limitations, High Performance Computing (HPC) techniques are required. Parallel computing enables multiple numbers to be tested for primality simultaneously, reducing overall execution time.

This project explores three different approaches:

- **Serial CPU Execution:** Checks numbers one at a time using a single thread
- **Parallel CPU Execution (Multithreading):**Utilizes multiple CPU cores to perform concurrent prime checking

Objective of the Project

The primary objective is not only to generate prime numbers but also to analyze and compare the performance of serial and parallel computational methods.

The project aims to:

- Reduce execution time for large numerical ranges
- Improve scalability using multithreading
- Demonstrate the advantages of parallel processing over serial execution

Real-World Relevance

Efficient prime number generation is important in several real-world applications such as:

- Cryptography and secure communication systems
- Data encryption and cybersecurity applications
- Mathematical research and scientific computing

Faster prime number computation improves system efficiency and supports applications that rely heavily on numerical processing and secure data operations.

3. Methodology / Procedure

The methodology of this project focuses on efficient prime number generation using parallel computing techniques. It integrates input processing, sequential execution, multithreaded CPU parallelization, and performance evaluation.

1. Input Preparation

The prime number generator operates based on a numerical range provided by the user during program execution.

The user specifies:

Starting value of the range Ending value of the range

All numbers within this range are considered candidates for prime number verification. This approach simulates computational workloads where large numerical datasets must be processed efficiently.

2. Prime Number Verification Logic

The core operation of this project is determining whether a given number is prime.

Each number is checked using divisibility testing:

$$Prime(n) = \{\ 1\ , if then number has no divisors other than 1 and itself 0, otherwise$$

This process is repeated for all numbers within the specified range.

3. Serial CPU Implementation

In the serial approach, numbers are processed sequentially using a single execution thread.

- Each number is checked one after another
- Simple and straightforward implementation
- No parallelism is utilized

4. Parallel CPU Implementation (Multithreading)

To improve performance, multithreading is used to parallelize prime number generation.

- The input range is divided into multiple subranges
- Each thread processes a subset of numbers
- Reduces execution time compared to serial approach

5. Thread Execution Strategy

Thread Mapping

Each thread is assigned a portion of the numerical range for prime verification.

If N is the total numbers in the given range and T is the number of threads:

$$N = End - Start + 1$$

$$ChunkSize = \left\lceil \frac{N}{T} \right\rceil$$

For thread i:

$$ThreadStart_i = Start + i \times ChunkSize$$

$$ThreadEnd_i = \min(ThreadStart_i + ChunkSize - 1,\ End)$$

Parallel Execution

Each thread performs the following operations:

- Reads numbers from its assigned range
- Performs prime number verification
- Stores identified prime numbers in a shared result structure

6. Performance Measurement

Execution time is measured for both implementations:

- Serial CPU execution using system time functions
- Parallel CPU execution using multithreading timing mechanisms

Performance comparison highlights execution time reduction achieved through parallel processing.

7. Complexity Analysis

The time complexity of prime number is:

$$O(n \times \sqrt{n})$$

where:

- n = total numbers in the given range
- $\sqrt{n}$ = number of divisibility checks required for each number

Parallel computing reduces execution time by distributing work across multiple processing units.

8. Overall Workflow

1. Accept numerical range as input
2. Generate numbers within the range
3. Perform matching using:
    - Serial CPU execution
    - Parallel CPU execution (Multithreading)
4. Measure execution time
5. Compare performance
6. Analyze efficiency improvement using parallel processing

4. Code

Project Source Code:

The complete project source code is available in the GitHub repository. This includes dataset generation, implementation, and performance analysis.

GitHub Link: https://github.com/2303A510G4-star/HPC-Project

4.1 Dataset Generation

```
%%writefile generate_numbers.py
limit = 100000

with open("Numbers.txt", "w") as f:
    for i in range(1, limit + 1):
        f.write(str(i) + "\n")

print("Numbers.txt created")
```

4.2 CPU CODE (Serial vs Parallel)

```
%%writefile cpu_prime_threads.c
#include <stdio.h>
#include <math.h>
#include <time.h>
#include <pthread.h>

#define MAX 200000
#define THREADS 4

int numbers[MAX];
int n = 0;
int count_parallel = 0;

typedef struct {
    int start;
    int end;
} ThreadData;

int isPrime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i <= sqrt(num); i++) {
        if (num % i == 0)
```

```
            return 0;
    }
    return 1;
}

void* findPrimes(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int local_count = 0;

    for (int i = data->start; i < data->end; i++) {
        if (isPrime(numbers[i]))
            local_count++;
    }

    __sync_fetch_and_add(&count_parallel, local_count);
    return NULL;
}

int main() {
    FILE *fp = fopen("Numbers.txt", "r");

    if (fp == NULL) {
        printf("Error opening Numbers.txt\n");
        return 1;
    }

    while (fscanf(fp, "%d", &numbers[n]) != EOF) n++;
    fclose(fp);

    printf("Total Numbers: %d\n", n);

    int repeat;
    printf("Enter repeat count: ");
    scanf("%d", &repeat);

    int count_serial = 0;

    // SERIAL
    clock_t s1 = clock();
    for (int r = 0; r < repeat; r++) {
        count_serial = 0;
        for (int i = 0; i < n; i++) {
            if (isPrime(numbers[i]))
                count_serial++;
        }
    }
    clock_t e1 = clock();

    double serial = (double)(e1 - s1) / CLOCKS_PER_SEC;

    // PARALLEL (THREADS)
```

```
    double s2 = (double)clock() / CLOCKS_PER_SEC;

    for (int r = 0; r < repeat; r++) {
        pthread_t threads[THREADS];
        ThreadData data[THREADS];
        count_parallel = 0;

        int chunk = n / THREADS;

        for (int i = 0; i < THREADS; i++) {
            data[i].start = i * chunk;
            data[i].end = (i == THREADS - 1) ? n : (i + 1) * chunk;

            pthread_create(&threads[i], NULL, findPrimes, &data[i]);
        }

        for (int i = 0; i < THREADS; i++) {
            pthread_join(threads[i], NULL);
        }
    }

    double e2 = (double)clock() / CLOCKS_PER_SEC;

    double parallel = e2 - s2;

    printf("Serial Time = %f sec\n", serial);
    printf("Parallel Time = %f sec\n", parallel);

    FILE *out = fopen("cpu_output.txt", "w");
    fprintf(out, "%f %f", serial, parallel);
    fclose(out);

    return 0;
}
```

4.3 GPU CODE (CUDA)

```
%%writefile gpu_prime.py
from numba import cuda
import numpy as np
import math
import time

@cuda.jit
def is_prime_gpu(nums, results):
    idx = cuda.grid(1)
    if idx < nums.size:
        n = nums[idx]
        if n < 2:
            results[idx] = 0
```

```
            return
        flag = 1
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                flag = 0
                break
        results[idx] = flag

nums = np.arange(1, 100000)
results = np.zeros_like(nums)

d_nums = cuda.to_device(nums)
d_results = cuda.to_device(results)

threads = 256
blocks = (nums.size + threads - 1) // threads

start = time.time()

is_prime_gpu[blocks, threads](d_nums, d_results)
cuda.synchronize()

end = time.time()

results = d_results.copy_to_host()
count = np.sum(results)

print("Total Primes:", count)
print("GPU Time:", (end - start), "sec")

with open("gpu_output.txt", "w") as f:
    f.write(str((end - start) * 1000))
```

4.4 Python Visualization Code

```
import matplotlib.pyplot as plt

with open("cpu_output.txt") as f:
    serial, parallel = map(float, f.read().split())

with open("gpu_output.txt") as f:
    gpu = float(f.read()) / 1000

print("\nFINAL RESULT")
print("Serial   :", serial, "sec")
print("Parallel :", parallel, "sec")
print("GPU      :", gpu, "sec")

methods = ['Serial', 'Parallel', 'GPU']
times = [serial, parallel, gpu]
```

```
16
17 plt.figure()
18 plt.bar(methods, times)
19
20 for i, v in enumerate(times):
21     plt.text(i, v, f"{v:.4f}", ha='center')
22
23 plt.title("Performance Comparison")
24 plt.ylabel("Time (sec)")
25 plt.xlabel("Execution Method")
26
27 plt.show()
```

5. Output Screenshots

CPU Results

```
Total Numbers: 100000
Enter repeat count: 5
Serial Time = 0.063661 sec
Parallel Time = 0.107454 sec
```

GPU Results

```
Total Primes: 9592
GPU Time: 0.4583868980407715 sec
```

Figure 5.1: GPU Execution Output for DNA Sequence Matching

FINAL RESULT

Method	Execution Time (seconds)
Serial	8.025631
Parallel	5.256038
GPU	0.105362

Comparison Graph

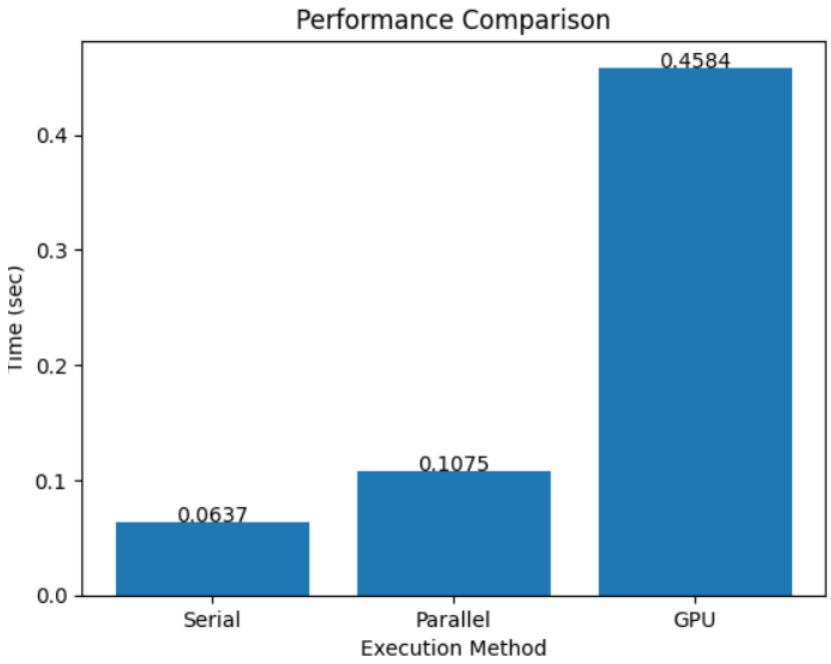

Figure 5.2: Comparison Graph

Figure 5.3: Performance Comparison of Serial CPU, Parallel CPU, and GPU Implementation

6. Observations, Limitations and Future Scope

7. Observations, Limitations and Future Scope

Observations

- GPU acceleration provides significant performance improvement compared to serial CPU execution for prime number generation.
- Parallel CPU implementation using OpenMP reduces execution time by distributing number checking across multiple processor cores.
- CUDA-based GPU implementation efficiently launches thousands of threads, allowing simultaneous primality testing for large ranges of numbers.
- Execution time decreases as the number range increases because parallel resources are better utilized.
- GPU computing proves highly effective for computationally intensive mathematical operations like large-scale prime number generation.

Limitations

- The current implementation uses basic primality testing, which may not be optimal for extremely large numbers.
- GPU memory capacity restricts the maximum range of numbers processed at a single time.
- Data transfer between CPU (host) and GPU (device) introduces additional overhead that affects performance for smaller inputs.
- Performance gain depends on hardware availability such as multicore CPUs or CUDA-enabled GPUs.
- Thread synchronization and load balancing issues may occur when workload distribution is uneven.

Future Scope

- Implement advanced prime generation algorithms such as the Sieve of Eratosthenes or Segmented Sieve for higher efficiency.
- Optimize CUDA kernels using shared memory and better memory access patterns.
- Extend implementation to support very large integer ranges using distributed computing.

- Integrate multi-GPU execution for handling massive numerical computations.
- Develop real-time visualization of prime generation performance and execution statistics.

Conclusion

This project successfully demonstrates the use of High Performance Computing techniques for parallel prime number generation. The comparison between serial CPU, parallel CPU, and GPU implementations clearly shows that parallel processing significantly reduces computation time.

Among all methods, the GPU implementation achieves the best performance due to its massive parallel execution capability. By distributing primality testing across thousands of threads, large numerical ranges can be processed efficiently.

Overall, the project highlights how parallel computing enhances computational efficiency for mathematically intensive problems. Such approaches are highly valuable in fields like cryptography, scientific computing, and large-scale numerical analysis.

8. References

- Prime Number – Wikipedia: https://en.wikipedia.org/wiki/Prime_number
- CUDA Samples (Parallel Computing Examples) : https://github.com/NVIDIA/cuda-samples
- OpenMP Examples (Parallel CPU Programming) : : https://github.com/OpenMP/Examples
- Sieve of Eratosthenes – Wikipedia : https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes
- Parallel Programming with CUDA – NVIDIA Developer Blog https://developer.nvidia.com/blog/tag/cuda/
- NVIDIA CUDA Toolkit Documentation :https://docs.nvidia.com/cuda/
- High Performance Computing Concepts – MIT OpenCourseWare: https://ocw.mit.edu/courses/parallel-computing/
- Google Colab: https://colab.research.google.com/drive/1lEys0KGshrom091qlf